Desde cero hasta ChatGPT 🤖

Día 3: Mecanismo de atención, Transformers, Pre-training y Fine-tuning

Ronghao Pan

2 de marzo de 2026

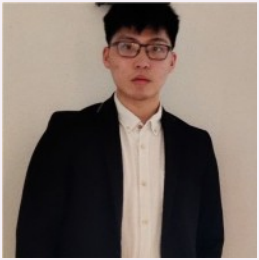
Sobre los docentes 👩‍🏫

José Antonio Hernández López

- Matemático e informático
- Doctor en Informática
- Profesor universitario de la UMU
- Investigación: LLMs para generación de código
- joseantonio.hernandez6@um.es

Ronghao Pan

- Ingeniero informático
- Doctorando e investigador en PLN
- Miembros del TECNOMOD y UMUTeam
- Investigación: LLMs, ASR, VLM
- ronghao.pan@um.es



Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT
- ...

Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT
- ...

pero... ¿sabéis...

- cómo funciona una red neuronal por dentro?
- de dónde viene la arquitectura transformer?
- cómo se ha llegado hasta ChatGPT?
- en qué se basan los agentes LLMs?

Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT
- ...

pero... ¿sabéis...

- cómo funciona una red neuronal por dentro?
- de dónde viene la arquitectura transformer?
- cómo se ha llegado hasta ChatGPT?
- en qué se basan los agentes LLMs?

El **objetivo del curso** es que comprendáis en profundidad el funcionamiento interno de las redes neuronales, yendo un paso más allá de lo que suele encontrarse en tutoriales o recursos generales de Internet. Seguiremos un enfoque progresivo, en el que iremos construyendo poco a poco todo el conocimiento necesario sobre redes neuronales hasta llegar a entender cómo funcionan modelos como ChatGPT.

Roadmap general 🗺️

1. Redes neuronales y Pytorch - **Día 1**
2. Redes recurrentes y tokenización - **Día 2**
3. Mecanismo de atención, Transformers, Pre-training y Fine-tuning - **Día 3**
4. Modelos decoder-only. De GPT a ChatGPT - **Día 4.1**
5. Agentes LLM - **Día 4.2**

Roadmap del día 3 🗺️

1. Arquitectura Transformer
   - Mini repaso - PLN
   - Arquitectura Transformers
   - Mecanismo de atención
   - Encoder-Decoder
2. Pre-training y fine-tuning
3. Modelos Encoder-only

Arquitectura Transformer

Mini repaso - PLN

Procesamiento de Lenguaje Natural (PLN)

Definición

El **PLN** es el conjunto de técnicas que permiten a las máquinas **entender, interpretar y generar** lenguaje humano.

- Campo clave de la IA y muy interdisciplinar: informática, lingüística, psicología, etc.
- Objetivo: reducir la brecha entre **lenguaje humano** y **lenguaje de máquina**.
- Ejemplos cotidianos: corrector, traductor, chatbots, asistentes de voz, resumen automático, detector de emociones, etc.

Procesamiento de Lenguaje Natural (PLN)

Ha habido tres grandes etapas de PLN:

- **1940s–1960s:** reglas y primeros intentos de traducción automática.
- **1970s–1990s:** simbolismo vs estadística; auge de modelos probabilísticos.
- **2000s–hoy:** aprendizaje automático y **deep learning**.

Hitos (orientativos): Tuning test (1950), Word2Vec(2013), BERT(2018), Transformers/GPT, LLMs

Procesamiento de Lenguaje Natural (PLN)

Las tareas de PLN van desde el preprocesamiento hasta la compresión y generación del lenguaje humano y las más importantes son:

- Segmentación y tokenización
- Análisis lingüístico (POS, NER, relaciones)
- Clasificación y comprensión
- Generación (resumen, traducción, QA)

Como se pueden observar, muchas tareas requieren no solo estructura superficial del texto, sino semántica, contexto y pragmática.

Segmentación de palabras (Tokenization)

Definición

La segmentación consiste en dividir un texto en unidades básicas (tokens).

Ejemplo en inglés: The cat sits on the mat.
Salida: [The | cat | sits | on | the | mat]

Ejemplo en español: Hoy hace muy buen tiempo.
Salida: [Hoy | hace | muy | buen | tiempo | .]

Importancia

La tokenización es el primer paso para POS, NER, parsing y clasificación.

Segmentación en subpalabras (Subword)

Motivación

Resolver palabras raras o desconocidas (Out-Of-Vocabulary).

Métodos comunes:

- BPE
- WordPiece
- Unigram
- SentencePiece

Ejemplo: `unhappiness`

- Sin subwords: [unhappiness]
- Con subwords: [un | happi | ness]

El modelo puede inferir significado combinando subunidades conocidas.

Etiquetado gramatical (POS Tagging)

Objetivo

Asignar una categoría gramatical a cada palabra.

Ejemplo:

She is playing the guitar in the park.

- She / PRP
- is / VBZ
- playing / VBG
- the / DT
- guitar / NN
- in / IN
- the / DT
- park / NN

Uso

Parsing, NER, traducción automática, análisis semántico.

Clasificación de texto

Objetivo

Asignar un texto a una o varias categorías predefinidas.

Ejemplos:

- "The team won the championship last night." $\rightarrow$ Deportes
- "The president announced new economic measures." $\rightarrow$ Política
- "Apple released a new MacBook with an M3 chip." $\rightarrow$ Tecnología

Uso

Análisis de sentimiento, detección de spam, clasificación de noticias.

Reconocimiento de entidades (NER)

Objetivo

Identificar entidades relevantes y clasificarlas.

Ejemplo:
John Smith works at Google in New York.
Salida:

- John Smith — PERSON
- Google — ORGANIZATION
- New York — LOCATION

Uso

Extracción de información, QA, sistemas de recomendación.

Extracción de relaciones

Objetivo

Identificar relaciones semánticas entre entidades.

Ejemplo:
Bill Gates founded Microsoft.
Salida:
(Bill Gates, founder_of, Microsoft)

Uso

Construcción de grafos de conocimientos (*knowledge graphs*) y sistemas de razonamiento.

Resumen automático

Objetivo

Generar un resumen breve y fiel del texto original.

Tipos:

- Extractivo: selecciona frases del texto
- Abstractivo: genera nuevas frases

Aplicaciones

Noticias, informes, motores de búsqueda.

Traducción automática

Objetivo

Traducir texto entre idiomas preservando el significado.

Ejemplo:
Spanish: Hoy hace muy buen tiempo.
English: The weather is very nice today.

Modelos modernos

Seq2Seq con atención y Transformers.

Pregunta–Respuesta (QA)

Objetivo

Responder preguntas formuladas en lenguaje natural.

Tipos:

- Retrieval-based QA
- Knowledge-based QA
- Community-based QA

Ejemplo

Q: Who founded Microsoft?
A: Bill Gates.

Representación de texto: de sparse a dense

Problema central

Las máquinas necesitan números: debemos convertir texto a **vectores**.

Etapa 1: VSM / One-Hot (sparse)

- Vocabulario grande $\Rightarrow$ vectores de alta dimensión
- Muy dispersos (casi todo ceros)
- Pierden **orden** y **contexto**

Ejemplo

Vocab=16384, frase con 5 palabras $\Rightarrow$ vector con 5 unos y 16379 ceros.

Modelos de lenguaje estadísticos: N-gram

Idea (Markov)

La probabilidad de una palabra depende solo de las $N-1$ anteriores.

$$P(w_1, \ldots, w_T) \approx \prod_{t=1}^{T} P(w_t \mid w_{t-(N-1)}, \ldots, w_{t-1})$$

- **Ventaja:** simple y útil
- **Limitación:** datos escasos cuando N crece (sparsidad)
- Captura mal dependencias largas y semántica compleja

Word2Vec: embeddings estáticos

Idea

Aprender vectores densos donde palabras con contextos similares están cerca.

- **CBOW:** contexto $\rightarrow$ palabra objetivo
- **Skip-gram:** palabra $\rightarrow$ contexto

Ventajas y limitaciones

- $+$ Reduce dimensionalidad, captura similitud semántica
- $-$ Un vector por palabra: no distingue polisemia (mismo vector para distintos sentidos)

ELMo: embeddings contextuales (dinámicos)

Idea clave

Un mismo token puede tener distintas representaciones según el contexto.

- Usa un **modelo de lenguaje** basado en **BiLSTM**
- **Preentrenamiento** en gran corpus + **extracción** de embeddings para tareas downstream

Motivación

Resolver la polisemia: *bank* (río) vs *bank* (finanzas).

Arquitectura Transformers

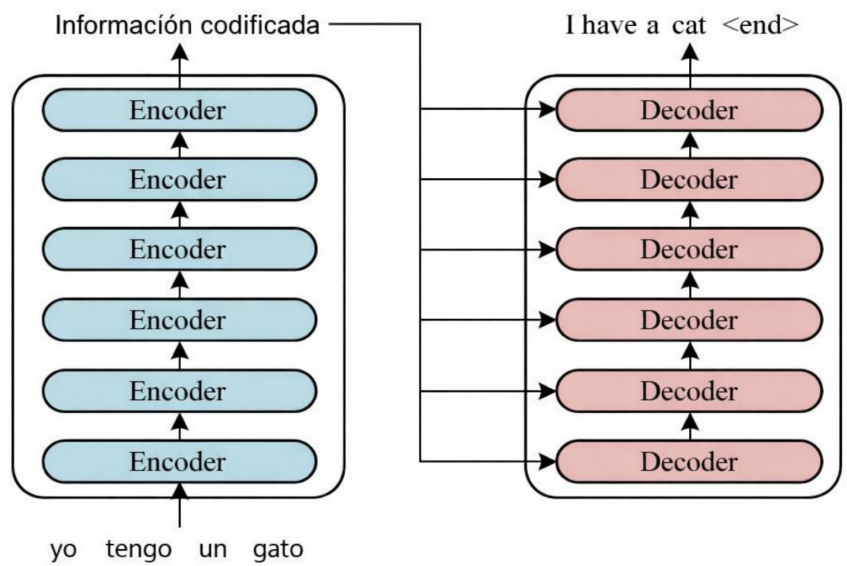
Arquitectura general de un Transformer

- Componentes principales:
  - **Tokenizer**: texto $\rightarrow$ índices de tokens.
  - **Token Embedding**: índices $\rightarrow$ vectores densos.
  - **Positional Encoding**: añade información de orden.
  - **Encoder**: N capas de Self-Attention + FNN.
  - **Decoder**: N capas de Masked Self-Attention + Cross-Attention + FNN.
  - **Capa de salida**: proyección al vocabulario + Softmax.
- Flujo típico:

$$\text{texto fuente} \rightarrow \text{Encoder} \rightarrow \text{Decoder} \rightarrow \text{texto objetivo}$$

- Diseño modular: casi todos los LLM actuales son variaciones de este esquema.

Arquitectura general de un Transformer

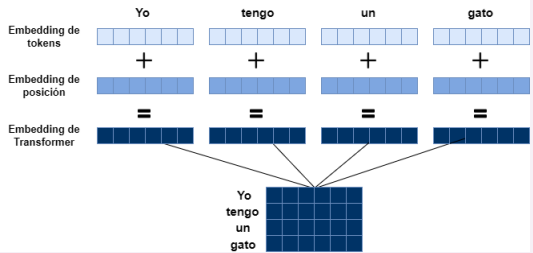


Flujo de trabajo de un Transformer

- **Paso 1: Representación de entrada**
  - Cada palabra se convierte en un vector mediante **Token Embedding**.
  - Se suma la **Positional Encoding** para incorporar información de orden.
  - Se obtiene la matriz de entrada:

$$X \in \mathbb{R}^{n \times d}$$

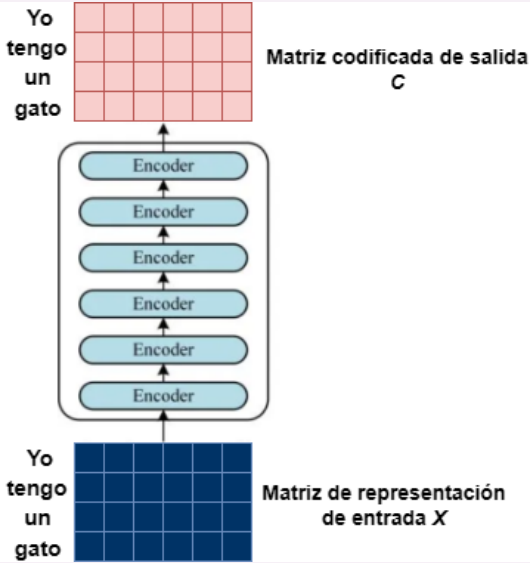
donde n es el número de tokens y d la dimensión del modelo (por ejemplo, $d = 512$).



Flujo de trabajo de un Transformer

- La matriz X se introduce en N **Encoder Blocks**.
- Cada bloque mantiene la dimensión $n \times d$.
- Se obtiene la representación codificada:

$$C = \text{Encoder}(X)$$

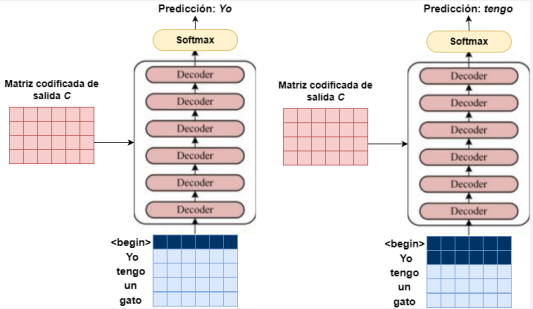


Flujo de trabajo de un Transformer

Paso 3: Decodificación (Decoder)

- La matriz C se pasa al **Decoder**.
- El Decoder genera la salida de forma autoregresiva.
- Se aplica **Masked Self-Attention** para evitar ver palabras futuras.
- Se predice la siguiente palabra:

$$y_{i+1} = \text{Decoder}(y_{\leq i}, C)$$



Embeddings: representar tokens

- La entrada al modelo son índices de tokens:

$$\text{idx} \in \mathbb{N}^{B \times T}$$

- La capa **Embedding** aprende una tabla:

$$E \in \mathbb{R}^{\text{vocab_size} \times d_{\text{model}}}$$

  donde cada fila es el vector de un token.
- Resultado:

$$X^{\text{emb}} = E(\text{idx}) \in \mathbb{R}^{B \times T \times d_{\text{model}}}$$

- Estos vectores capturan similitudes semánticas (tokens con significado parecido tienden a tener embeddings similares).

Positional Encoding: añadir el orden

- La atención por sí sola no distingue el orden de los tokens.
- El **Positional Encoding** añade un vector $PE(pos)$ a cada posición:

$$X_t = X_t^{\text{emb}} + PE(t)$$

- En el Transformer original se usa codificación **sinusoidal**:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- Propiedades:
  - Generaliza a longitudes mayores que las del entrenamiento.
  - Codifica información de posición **absoluta y relativa**.

Mecanismo de atención

¿Qué es un mecanismo de atención?

Motivación

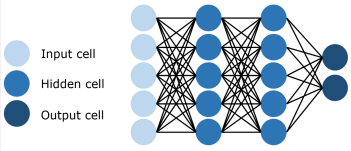
Con el paso del PLN desde métodos estadísticos a deep learning, la representación del texto pasó a aprenderse mediante redes neuronales.

- De modelos basados en conteo y reglas
- A modelos que aprenden representaciones automáticamente

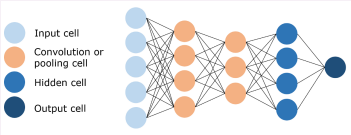
Las redes neuronales profundas se desarrollaron inicialmente en tres grandes familias:

- **Redes totalmente conectadas (FNN)**
- **Redes convolucionales (CNN)**
- **Redes recurrentes (RNN)**

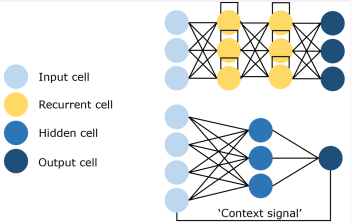
Arquitecturas neuronales clásicas



FNN



CNN



RNN

FNN y CNN en PLN

Limitaciones

FNN:

- No modelan secuencias ni orden temporal
- Tratan el texto como un vector fijo

CNN:

- Capturan patrones locales (n-gramas)
- Dificultad para relaciones de largo alcance

El lenguaje es secuencial $\rightarrow$ se necesitan modelos secuenciales.

RNN y LSTM: dominio del PLN clásico

Por qué dominaron el PLN

Las RNN están diseñadas para procesar secuencias y datos temporales.

- Mantienen un estado que resume el pasado
- Adecuadas para generación de texto
- LSTM introduce compuertas para mejorar la memoria

Presentan una serie de limitaciones:

- **Limitación 1: cálculo secuencial**
- **Limitación 2: dependencias a largo plazo**

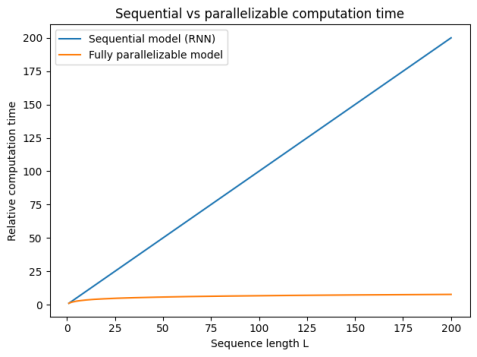
Limitación 1: cálculo secuencial

Problema computacional

Las RNN procesan la secuencia paso a paso:

- Cada token depende del anterior
- No se puede paralelizar fácilmente
- El uso de GPU es ineficiente

Entrenamiento lento incluso con modelos relativamente pequeños.

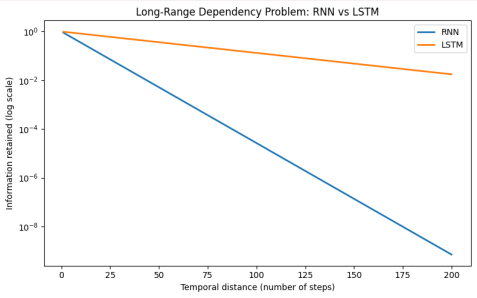


Limitación 2: dependencias a largo plazo

Problema de memoria

- Las relaciones lejanas son difíciles de capturar
- El estado oculto es un cuello de botella
- LSTM mitiga el problema, pero no lo elimina

El modelo pierde información relevante de tokens distantes.



Mecanismo de atención

Definición

El mecanismo de atención permite que un modelo:

- no procese toda la información por igual,
- se concentre en las partes más relevantes de la entrada.

Origen

La atención surge en **visión por computador**:

- al observar una imagen, no miramos cada píxel con la misma importancia,
- nos enfocamos en las regiones clave.

En PLN, el mismo principio se aplica a los **tokens**.

Atención en Procesamiento del Lenguaje Natural

Motivación

En una secuencia de texto:

- no todas las palabras son igual de importantes,
- la relevancia depende del objetivo actual.

Ejemplo

Para responder a la pregunta:

"¿Cuándo ocurrió la noticia?"

El modelo debe prestar más atención a palabras relacionadas con:

- fechas,
- tiempo,
- eventos temporales.

Query, Key y Value

El mecanismo de atención se basa en tres componentes:

- **Query (Q)**: lo que estamos buscando
- **Key (K)**: cómo se describen los elementos disponibles
- **Value (V)**: la información asociada a cada Key

Idea clave

La atención calcula:

qué tan relevante es cada Key para una Query

y usa esa relevancia para combinar los Value.

Ejemplo intuitivo: diccionario

Supongamos el siguiente diccionario:

$$\{\texttt{"apple"} : 10,\ \texttt{"banana"} : 5,\ \texttt{"chair"} : 2\}$$

- Keys $\rightarrow$ `apple, banana, chair`
- Values $\rightarrow$ `10, 5, 2`

Consulta exacta

Si la Query es `"apple"`, recuperamos directamente el valor 10.

Consulta semántica

Si la Query es `"fruit"`:

- `apple` y `banana` son relevantes,
- `chair` no lo es.

Asignamos pesos (atención):

$$\{\texttt{"apple"} : 0.6,\ \texttt{"banana"} : 0.4,\ \texttt{"chair"} : 0\}$$

Resultado

$$0.6 \cdot 10 + 0.4 \cdot 5 + 0 \cdot 2 = 8$$

Atención como similitud vectorial

Embeddings

Las palabras se representan como vectores densos que codifican semántica.

Medida de similitud

Usamos el producto punto:

$$v \cdot w = \sum_i v_i w_i$$

- palabras semánticamente cercanas $\rightarrow$ producto grande,
- palabras no relacionadas $\rightarrow$ producto pequeño o negativo.

Cálculo de atención

Sea:

- q la Query,
- $K = [k_1, k_2, \ldots, k_n]$ la matriz de Keys.

Scores

$$x = qK^T$$

Normalización

$$\alpha_i = \text{softmax}(x)_i$$

Los α_i representan los **pesos de atención**.

Fórmula general de atención

Una sola Query

$$\text{Attention}(q, K, V) = \text{softmax}(qK^T)V$$

Múltiples Queries

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T)V$$

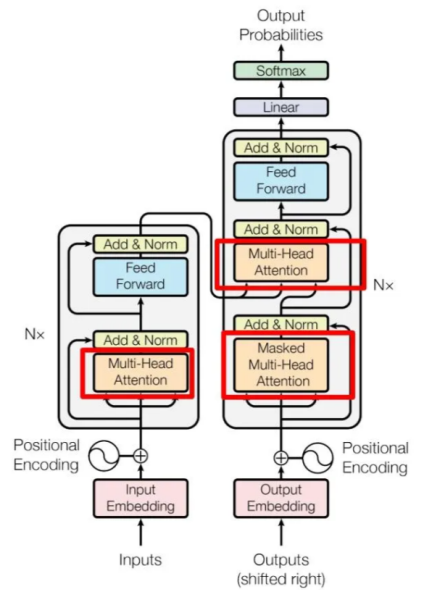
Cuando la dimensión d_k es grande:

- los valores de QK^T crecen,
- softmax se vuelve inestable.

Solución

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Mecanismo de atención en Transformer



Tipos de mecanismo de atención: Self-Attention

Definición

En self-attention:

$$Q = K = V = X$$

Cada token:

- consulta a todos los demás tokens,
- aprende relaciones internas de la secuencia.

Ejemplo

Una palabra puede atender a:

- su sujeto,
- su objeto,
- modificadores lejanos.

Tipos de mecanismo de atención: Self-Attention

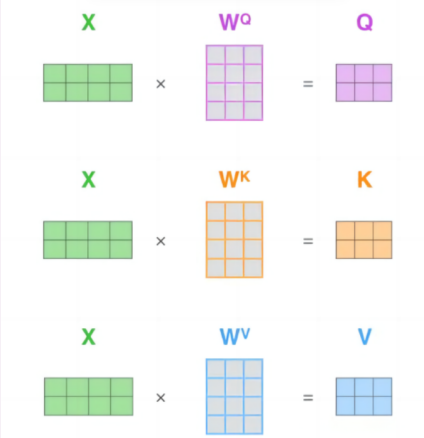


Figura: Obtener valores Q, K y V

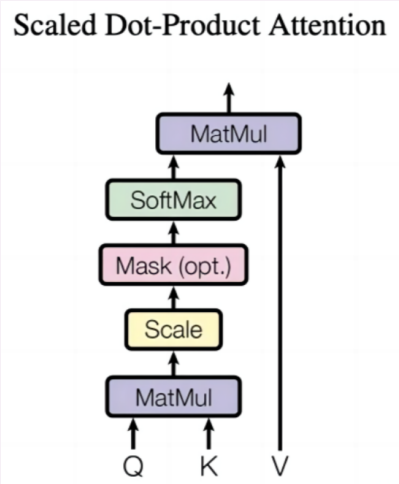


Figura: Arquitectura de self-attention

Pytorch: Self-Attention

```
class Self_Attention(nn.Module):
    # input : batch_size * seq_len * input_dim
    # q : batch_size * input_dim * dim_k
    # k : batch_size * input_dim * dim_k
    # v : batch_size * input_dim * dim_v
    def __init__(self, input_dim, dim_k, dim_v):
        super(Self_Attention, self).__init__()
        self.q = nn.Linear(input_dim, dim_k)
        self.k = nn.Linear(input_dim, dim_k)
        self.v = nn.Linear(input_dim, dim_v)
        self.norm_fact = 1 / sqrt(dim_k)
    def forward(self, x):
        Q = self.q(x)  # Q: batch_size * seq_len * dim_k
        K = self.k(x)  # K: batch_size * seq_len * dim_k
        V = self.v(x)  # V: batch_size * seq_len * dim_v
        # Q * K^T / sqrt(dim_k)
        atten = nn.Softmax(dim=-1)(
            torch.bmm(Q, K.permute(0, 2, 1)) * self.norm_fact
        )
        # attention * V
        output = torch.bmm(atten, V)
        return output
```

Tipos de mecanismo de atención: Masked Self-Attention

Definición

La **autoatención enmascarada** es un mecanismo de atención que utiliza una **máscara** para ocultar ciertos tokens durante el cálculo de atención.

- Evitar que el modelo vea información futura.
- Garantizar predicción **causal** (solo pasado).
- Permitir entrenamiento **paralelo**.

Implementación

Se usa una matriz triangular superior con $-\infty$:

$$\text{scores} = QK^T + \text{mask}$$

Ejemplos: Masked Self-Attention

Se construyen múltiples ejemplos de entrenamiento en paralelo, pero se **oculta el futuro** mediante una máscara. Por ejemplo, una entrada del modelo:

```
<BOS> [MASK] [MASK] [MASK] [MASK]
<BOS>    I   [MASK] [MASK] [MASK]
<BOS>    I     like [MASK] [MASK]
<BOS>    I     like   you  [MASK]
<BOS>    I     like   you  </EOS>
```

Cada fila solo ve el pasado, pero todas se procesan a la vez. Así pues, se puede entender la máscara como una **matriz triangular superior**:

- parte superior: $-\infty$ (oculta)
- diagonal e inferior: 0 (visible)

Si la entrada es:

$$(\mathit{batch_size},\ \mathit{seq_len},\ \mathit{hidden_size})$$

la máscara suele ser:

$$(1,\ \mathit{seq_len},\ \mathit{seq_len})$$

PyTorch: Creación de máscara

```
# Crear una matriz triangular superior para ocultar el futuro
mask = torch.full(
    (1, max_seq_len, max_seq_len),
    float("-inf")
)

# triu crea la parte triangular superior
mask = torch.triu(mask, diagonal=1)
```

- $-\infty$ bloquea la atención
- 0 permite la atención

PyTorch: Uso de la máscara en atención

```
# scores: QK^T / sqrt(d_k)
scores = scores + mask[:, :seqlen, :seqlen]

# Softmax
scores = F.softmax(scores.float(), dim=-1)
```

Efecto

- las posiciones futuras $\rightarrow 0$ tras softmax
- el modelo ignora completamente el futuro

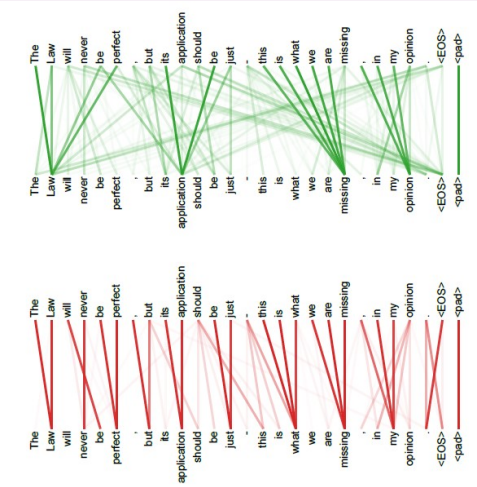
Tipos de mecanismo de atención: Multihead-Attention

Limitación de una sola atención

Una única operación de atención suele capturar **un solo tipo de relación** (sintáctica, semántica, coreferencia, dependencia a larga distancia, etc.).

Multi-Head Attention ejecuta varias atenciones en paralelo:

- cada **cabeza** aprende un patrón distinto de relaciones,
- luego se **concatena** la información y se proyecta a la dimensión del modelo.



Tipos de mecanismo de atención: Multihead-Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

donde cada cabeza es:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- h: número de cabezas
- W_i^Q, W_i^K, W_i^V: proyecciones aprendibles por cabeza
- W^O: proyección final tras concatenación

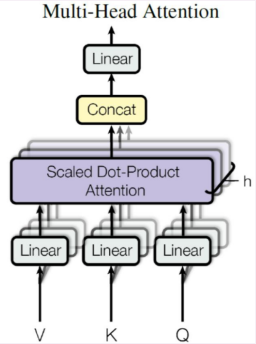


Figura: Arquitectura de multi-head attention

Multihead-Attention: Cómo se calcula (resumen por pasos)

1. Proyectar entradas: $Q = XW^Q,\ K = XW^K,\ V = XW^V$
2. Separar en h cabezas (reshape)
3. Calcular atención en cada cabeza:

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

4. Concatenar salidas de las cabezas
5. Proyectar: $\text{Concat}(\cdot)\, W^O$

Nota

Si es **causal**, se aplica máscara triangular para ocultar el futuro.

PyTorch: Multi-Head Self-Attention

```python
import ...
class MultiHeadAttention(nn.Module):
    def __init__(self, dim, n_heads, dropout=0.0, is_causal=False, max_seq_len=2048):
        super().__init__()
        assert dim % n_heads == 0
        self.dim = dim
        self.n_heads = n_heads
        self.head_dim = dim // n_heads
        self.is_causal = is_causal

        self.wq = nn.Linear(dim, dim, bias=False)
        self.wk = nn.Linear(dim, dim, bias=False)
        self.wv = nn.Linear(dim, dim, bias=False)
        self.wo = nn.Linear(dim, dim, bias=False)

        self.attn_dropout = nn.Dropout(dropout)
        self.resid_dropout = nn.Dropout(dropout)

        if is_causal:
            mask = torch.full((1, 1, max_seq_len, max_seq_len), float("-inf"))
            mask = torch.triu(mask, diagonal=1)
            self.register_buffer("mask", mask)
```

PyTorch: Multi-Head Self-Attention

```python
def forward(self, x):
    B, T, _ = x.shape
    q = self.wq(x)
    k = self.wk(x)
    v = self.wv(x)

    # (B,T,dim) -> (B,nh,T,hd)
    q = q.view(B, T, self.n_heads, self.head_dim).transpose(1, 2)
    k = k.view(B, T, self.n_heads, self.head_dim).transpose(1, 2)
    v = v.view(B, T, self.n_heads, self.head_dim).transpose(1, 2)
    scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)  # (B,nh,T,T)

    if self.is_causal:
        scores = scores + self.mask[:, :, :T, :T]
    attn = F.softmax(scores.float(), dim=-1).type_as(scores)
    attn = self.attn_dropout(attn)
    out = attn @ v  # (B,nh,T,hd)

    # (B,nh,T,hd) -> (B,T,dim)
    out = out.transpose(1, 2).contiguous().view(B, T, self.dim)
    out = self.resid_dropout(self.wo(out))
    return out
```

Encoder-Decoder

Seq2Seq y arquitectura Encoder–Decoder

- **Seq2Seq (Sequence-to-Sequence)**:

$$\text{input} = (x_1, x_2, \ldots, x_n), \quad \text{output} = (y_1, y_2, \ldots, y_m)$$

- Muchos problemas de PLN se formulan como Seq2Seq:
  - Traducción automática
  - Resumen automático
  - Clasificación (salida de longitud 1)
  - Etiquetado secuencial (salida de longitud n)
- En Transformer:
  - **Encoder**: codifica la secuencia de entrada en representaciones semánticas.
  - **Decoder**: decodifica esas representaciones en la secuencia objetivo.

Transformer como modelo Encoder–Decoder

- El Transformer propuesto en *Attention Is All You Need*:
  - Usa **solo atención** (sin RNN ni CNN).
  - Está formado por:
    - Un **Encoder** con N capas (EncoderLayers).
    - Un **Decoder** con N capas (DecoderLayers).
- Flujo:
  1. La secuencia fuente entra al Encoder y se transforma en un conjunto de vectores ocultos.
  2. Esos vectores se pasan a cada capa del Decoder como **Key/Value** de la atención cruzada.
  3. El Decoder genera la secuencia objetivo, un token cada vez (auto-regresivo).
- Muchos modelos actuales:
  - Solo Encoder: **BERT**.
  - Solo Decoder: **GPT**.

Componentes de Encoder–Decoder: Feed-Forward Network (FNN)

- Cada capa del Transformer incluye un **Feed-Forward Network** (FNN) independiente para cada posición (position-wise).
- Estructura típica:

$$\mathrm{FFN}(x) = \mathrm{Dropout}\big(W_2\,\sigma(W_1 x)\big)$$

  donde:
  - W_1: proyección de `dim` a `hidden_dim`.
  - W_2: proyección de `hidden_dim` de vuelta a `dim`.
  - σ: función de activación, típicamente ReLU.
- Intuición:
  - La atención mezcla información entre posiciones.
  - El FNN aplica una transformación no lineal rica a cada posición.
  - Dropout ayuda a evitar sobreajuste.

PyTorch: Feed-Forward Network (FNN)

```python
class MLP(nn.Module):
    """Feed-Forward Network (position-wise)"""
    def __init__(self, dim: int, hidden_dim: int, dropout: float):
        super().__init__()
        # Primera capa lineal: dim -> hidden_dim
        self.w1 = nn.Linear(dim, hidden_dim, bias=False)
        # Segunda capa lineal: hidden_dim -> dim
        self.w2 = nn.Linear(hidden_dim, dim, bias=False)
        # Dropout para evitar sobreajuste
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # FFN: Linear -> ReLU -> Linear -> Dropout
        x = self.w1(x)
        x = F.relu(x)
        x = self.w2(x)
        x = self.dropout(x)
        return x
```

Componentes de Encoder–Decoder: Layer Normalization

- La normalización estabiliza la distribución de activaciones y facilita el entrenamiento de redes profundas.
- **BatchNorm** normaliza sobre el mini-batch (problemático con secuencias de distinta longitud, RNN, etc.).
- **LayerNorm** normaliza:
  - por **cada muestra** y sobre **todas las dimensiones** de la última capa.
- Para un vector $x \in \mathbb{R}^d$:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma + \varepsilon} + \beta$$

- γ y β son parámetros aprendibles (escala y sesgo).

PyTorch: Layer Normalization

```python
class LayerNorm(nn.Module):
    """Layer Normalization"""
    def __init__(self, features, eps=1e-6):
        super().__init__()
        # Parámetros aprendibles (escala y sesgo)
        self.a_2 = nn.Parameter(torch.ones(features))
        self.b_2 = nn.Parameter(torch.zeros(features))
        self.eps = eps

    def forward(self, x):
        # Media y desviación estándar sobre la última dimensión
        mean = x.mean(-1, keepdim=True)  # [B, T, 1]
        std = x.std(-1, keepdim=True)    # [B, T, 1]
        # Normalización + reescalado
        return self.a_2 * (x - mean) / (std + self.eps) + self.b_2
```

Componentes de Encoder–Decoder: Residual Connections

- Transformer es un modelo profundo (muchas capas).
- Sin atajos, las redes profundas sufren:
  - degradación del gradiente,
  - dificultad para optimizar.
- Idea de **residual connection**:

$$\text{out} = x + \text{sublayer}(\text{LayerNorm}(x))$$

- Beneficios:
  - El gradiente puede fluir directamente por el camino de la identidad.
  - Las capas superiores aprenden una *corrección* (residuo) sobre x.

PyTorch: Residual + LayerNorm

```
# x: tensor de entrada
# attention: MultiHeadAttention
# feed_forward: MLP
# attention_norm, ffn_norm: LayerNorm

# Subcapa 1: Self-Attention con residual
norm_x = attention_norm(x)
h = x + attention(norm_x)

# Subcapa 2: Feed-Forward con residual
norm_h = ffn_norm(h)
out = h + feed_forward(norm_h)
```

Encoder

- Entrada: secuencia fuente

$$(x_1, x_2, \ldots, x_{L_{\text{src}}})$$

- Cada token se convierte en:

$$z_i = \text{Embedding}(x_i) + \text{PositionalEncoding}(i)$$

- El Encoder apila N capas idénticas:
  1. Multi-Head Self-Attention (sin máscara causal).
  2. Residual + LayerNorm.
  3. FNN posición a posición.
  4. Residual + LayerNorm.
- Salida del Encoder:

$$H_{\text{enc}} \in \mathbb{R}^{L_{\text{src}} \times d_{\text{model}}}$$

que se usará como Key/Value en el Decoder.

Encoder

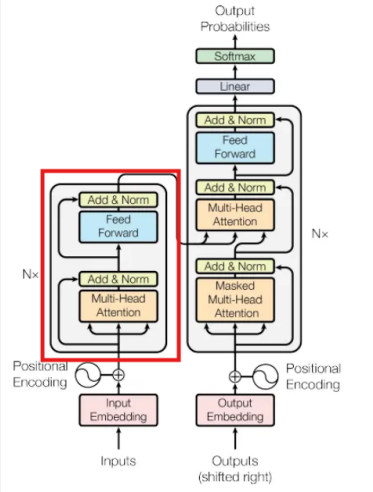


Figura: Arquitectura de encoder dentro de Transformer

PyTorch: EncoderLayer

```python
class EncoderLayer(nn.Module):
    """Encoder layer: Self-Attention + FFN"""
    def __init__(self, args):
        super().__init__()
        # LayerNorm antes de la Self-Attention
        self.attention_norm = LayerNorm(args.n_embd)
        # Self-Attention sin máscara causal
        self.attention = MultiHeadAttention(
            dim=args.n_embd,
            n_heads=args.n_heads,
            dropout=args.dropout,
            is_causal=False,
            max_seq_len=args.max_seq_len,
        )
        # LayerNorm antes del MLP
        self.fnn_norm = LayerNorm(args.n_embd)
        self.feed_forward = MLP(
            dim=args.n_embd,
            hidden_dim=args.ffn_hidden_dim,
            dropout=args.dropout,
        )
    def forward(self, x):
        # Self-Attention con residual
        norm_x = self.attention_norm(x)
        h = x + self.attention(norm_x)
```

PyTorch: Encoder

```python
class Encoder(nn.Module):
    """Encoder: stack of N EncoderLayers"""
    def __init__(self, args):
        super().__init__()
        self.layers = nn.ModuleList(
            [EncoderLayer(args) for _ in range(args.n_layer)]
        )
        self.norm = LayerNorm(args.n_embd)

    def forward(self, x):
        # Paso por N capas de Encoder
        for layer in self.layers:
            x = layer(x)
        # Normalización final
        return self.norm(x)
```

Decoder

- Entrada: secuencia objetivo desplazada

$$(\langle \text{BOS} \rangle, y_1, \ldots, y_{t-1})$$

- Cada capa del Decoder contiene:
  1. **Masked Multi-Head Self-Attention**:
     - máscara causal: cada posición sólo ve tokens anteriores.
  2. Residual + LayerNorm.
  3. **Cross-Attention**:
     - Query: estados del Decoder.
     - Key/Value: salidas del Encoder H_{enc}.
  4. Residual + LayerNorm.
  5. **FNN** posición a posición.
  6. Residual + LayerNorm.
- Salida del Decoder:

$$H_{\text{dec}} \rightarrow \text{capa de salida (vocabulario)}$$

Decoder

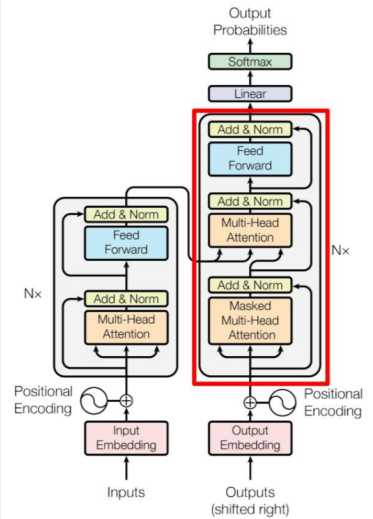


Figura: Arquitectura de decoder dentro de Transformer

PyTorch: DecoderLayer

```python
class DecoderLayer(nn.Module):
    """Decoder layer: Masked Self-Attn + Cross-Attn + FFN"""
    def __init__(self, args):
        super().__init__()
        # 1) Masked Self-Attention
        self.attention_norm_1 = LayerNorm(args.n_embd)
        self.mask_attention = MultiHeadAttention(
            dim=args.n_embd,
            n_heads=args.n_heads,
            dropout=args.dropout,
            is_causal=True,              # máscara causal
            max_seq_len=args.max_seq_len,
        )
        self.attention_norm_2 = LayerNorm(args.n_embd)
        self.attention = MultiHeadAttention(
            ...
        )
        # 3) Feed-Forward
        self.ffn_norm = LayerNorm(args.n_embd)
        self.feed_forward = MLP(
            ...
        )
```

```python
def forward(self, x, enc_out):
    # Masked Self-Attention
    norm_x = self.attention_norm_1(x)
    x = x + self.mask_attention(norm_x)
    # Cross-Attention
    norm_x = self.attention_norm_2(x)
    h = x + self.attention(norm_x, enc_o
    # Feed-Forward
    norm_h = self.ffn_norm(h)
    out = h + self.feed_forward(norm_h)
    return out
```

PyTorch: Decoder

```python
class Decoder(nn.Module):
    """Decoder: stack of N DecoderLayers"""
    def __init__(self, args):
        super().__init__()
        self.layers = nn.ModuleList(
            [DecoderLayer(args) for _ in range(args.n_layer)]
        )
        self.norm = LayerNorm(args.n_embd)

    def forward(self, x, enc_out):
        # Paso secuencial por N capas de Decoder
        for layer in self.layers:
            x = layer(x, enc_out)
        return self.norm(x)
```

Resumen: Encoder–Decoder en Transformer

- **Encoder**:
  - Self-Attention sin máscara + FNN + Residual + LayerNorm.
  - Produce representaciones contextualizadas de la entrada.
- **Decoder**:
  - Masked Self-Attention (auto-regresivo).
  - Cross-Attention sobre el Encoder.
  - FNN + Residual + LayerNorm.
- Juntos forman un modelo **Seq2Seq** totalmente basado en atención, eficiente y altamente paralelizable.

Pre-training y fine-tuning

Modelos task-specific: definición

En el deep learning, al principio, se llevaba los modelos entrenados desde cero para una tarea en concreto (*task-specific models*)

Task-specific models

Dada una tarea con un dataset asociado, los investigadores:

- Proponían una arquitectura f_θ definida en PyTorch (usualmente las arquitecturas ya conocidas pero algo modificadas)
- Lo entrenaban con el dataset desde cero
- Lo evaluaban con la porción de test de ese dataset

Ejemplos:

- CNN desde cero para clasificación de señales de tráfico
- LSTM desde cero para clasificación de textos
- Transformer desde cero para traducir textos
- etc.

Modelos task-specific: problemas

Problemas de los modelos *task-specific*:

- Generalización limitada: si cambias el test, la *performance* decae mucho...
- Etiquetar es costoso: hay que etiquetar muchos datos para entrenar redes profundas
- Poca transferencia: lo aprendido en una tarea no se reutiliza

Cambio de paradigma

Pre-training y fine-tuning

En este paradigma hay dos fases:

- **Pre-training.** Se parte un red neuronal profunda virgen y se pre-entrena usando alguna *pre-training task*: MLM, CLM, FIM, etc.
- **Fine-tuning.** Se parte de una red pre-entrenada y, usando un dataset potencialmente (aunque no necesariamente) etiquetado, se entrenan todos los parámetros en esa tarea específica.

Analogía en NLP. El pre-training es para dar nociones básicas sobre lenguaje natural y el fine-tuning es una intensificación en una tarea específica.

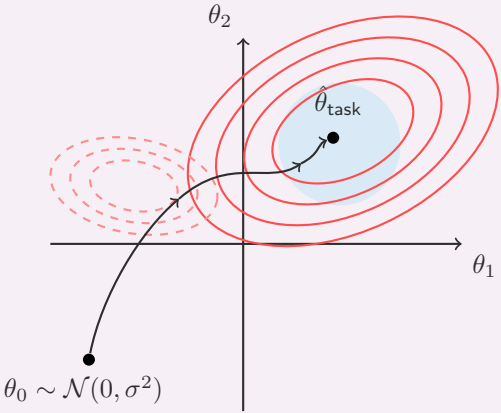
- BERT y GPT-1: transformer pre-entrenado para ser fine-tuneado en problemas de clasificación en lenguaje natural
- ResNet: CNN pre-entrando para ser fine-tuneado tareas con imágenes

Pre-training y fine-tuning

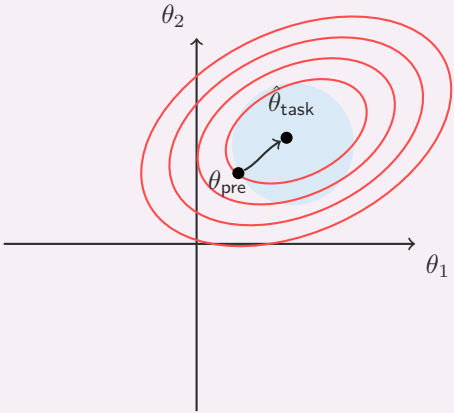
Pre-training y fine-tuning solucionan los problemas de los modelos *task-specific*:

- Mejor generalización: los modelos fine-tuneados suelen generalizar mejor
- Etiquetar no es tan costoso: los datasets fine-tuneados no tienen que ser tan grandes
- Mucha transferencia: un modelo pre-entrenado se puede fine-tunear en varias tareas

Pre-training y fine-tuning: intuición matemática



Entrenar **desde random**
(más pasos, mayor riesgo de mala región)



Fine-tuning desde **pre-entrenado**
(mejor inicialización, menos pasos)

Pre-training tasks

En procesamiento de lenguaje natural hay varias tareas de pre-entrenamieneto:

- Masked Language Modeling (MLM): dada una frase, de manera aleatoria se *maskean* varias palabras. La tarea consiste en predecir esas palabras
- Next Sentence Prediction (NSP): dadas dos frases, predecir si una sigue a la otra
- Casual Language Modeling (CLM): predecir el siguiente token dados los anteriores
- Fill-in-the-middle (FIM): generar un fragmento intermedio condicionado por prefijo + sufijo
- ...

Modelos Encoder-only

Origen del Transformer

Contexto histórico

El **Transformer** surge originalmente como una arquitectura **encoder–decoder** para tareas de **traducción automática**.

- Propuesto en 2017 en *"Attention is All You Need"*
- Diseñado para mapear:

$$\text{secuencia de entrada} \rightarrow \text{secuencia de salida}$$

Idea

El Transformer completo = Encoder + Decoder

Factores clave

Observación empírica

Varios *papers* demostraron que, en modelos encoder–decoder (no solo transformers, también RNNs), el **encoder** aprende representaciones ricas del texto de entrada.

- Captura semántica y sintaxis
- Codifica dependencias a larga distancia
- Sus estados internos pueden reutilizarse

El encoder actúa como extractor de características del lenguaje

Pre-training y fine-tuning

El paradigma pre-training y fine-tuning se había vuelto muy popular en visión (AlexNet e ImageNet).

- En NLP, no había un equivalente a ImageNet que funcionara tan bien

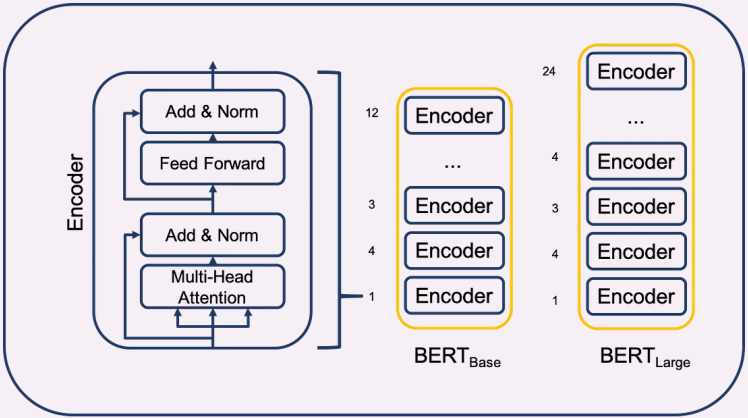
BERT: idea principal

Idea central

En 2018, BERT combina explícitamente esas dos ideas ya existentes y posibilita el pre-training a gran escala mediante un pre-training no supervisado de un encoder (sin el decoder).

- Pre-training de solo un encoder:
  - Encoder con varias capas transformer
  - Tare Masked Language Modeling (MLM): algunos tokens de las frases de entrada se enmascaran y el modelo debe predecirlos
  - Tarea Next Sentence Prediction (NSP): dadas dos frases, ¿una es la continuación de la otra?
- Fine-tuning:
  - Añadir una capa lineal en la salida
  - Entrenar el encoder a una tarea concreta (usualmente clasificación a nivel de token o de frase)

BERT: arquitectura



BERT: funcionamiento

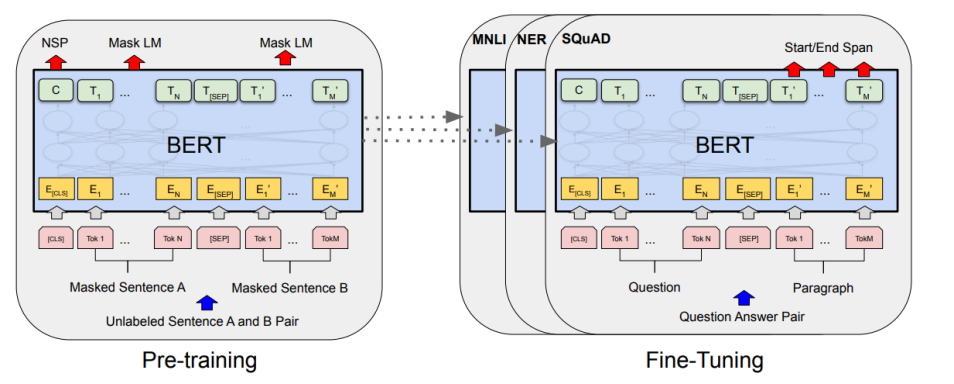


Figura: Funcionamiento de BERT extraído del paper original

BERT: resultados

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

Figura: Resultados de BERT extraídos del paper original

- BERT lideró varios bencharks con un gran margen

BERT: algunos detalles

En el paper original se presentaron dos modelos:

Modelo	**# Parámetros**	**Hidden size**	**# Layers**
BERT Base	∼110M	768	12
BERT Large	∼340M	1024	24

Cuadro: Comparación entre BERT Base y BERT Large

Los datasets usados fueron:

- Wikipedia (en inglés) - 2.500M de palabras
- BooksCorpus - 800M de palabras

En total, los modelos fueron entrenados con 3.300M de palabras

RoBERTa: idea principal

Ideas

- Los investigadores de RoBERTa consideraron que muchas de las decisiones del entrenamiento de BERT no tenían *fuste*...
- RoBERTa mantiene la **misma arquitectura que BERT** y se centra en optimizar el **pre-training**.

- Mismo encoder Transformer
- Mismo esquema de fine-tuning
- Cambios clave en:
  - datos
  - objetivos
  - hiperparámetros

RoBERTa = *Robustly Optimized BERT Approach*

RoBERTa: datos

Datos de pre-training

- Wikipedia
- BooksCorpus
- CommonCrawl (filtrado)
- News
- OpenWebText

$\sim$160 GB de texto, $\sim$30B tokens ($\sim$10$\times$ BERT)

RoBERTa: tareas de pre-training

Cambios importantes

- Se elimina **Next Sentence Prediction (NSP)**
- Se mantiene únicamente:
  - Masked Language Modeling (MLM)
- Se usa **dynamic masking**: cada vez que una frase aparece se enmascaran tokens distintos

RoBERTa: hiperparámetros

- **Arquitectura**:
  - BERT: encoder Transformer
  - RoBERTa: **idéntica a BERT**
- **Batch size**:
  - BERT: pequeño / moderado
  - RoBERTa: **muy grande** (hasta 8k secuencias, vía acumulación)
- **Learning rate**:
  - BERT: $\sim 10^{-4}$ (poco explorado)
  - RoBERTa: **ajustado empíricamente**
- **Entrenamiento**:
  - BERT: relativamente corto
  - RoBERTa: **más pasos y más epochs**

Los cambios de RoBERTa se centran en hacer el entrenamiento más estable

Conclusión: BERT y RoBERTa

- BERT y RoBERTa introduce el paradigma moderno de **encoder-only**
- Ambos modelos se siguen utilizando ampliamente hoy en día:
  - clasificación de textos
  - NER y tareas de token-level
  - recuperación de información
- Muchas arquitecturas posteriores se basan directa o indirectamente en estas ideas
  - XLM-R: extensión multilingüe de RoBERTa
  - DeBERTa: mejora la atención y las representaciones posicionales
  - ELECTRA: nuevo objetivo de pre-training sobre un encoder

Ejemplo práctico

- Tarea (para fine-tunear): análisis de sentimiento (positivo o negativo)
  - "Este producto es excelente" $\rightarrow$ positivo
  - "No me ha gustado nada, horrible" $\rightarrow$ negativo
- Dataset: `https://huggingface.co/datasets/mteb/SpanishSentimentClassification`
- Modelo pre-entrenado: `https://huggingface.co/BSC-LT/mRoBERTa`

En la práctica:

1. Al modelo pre-entrenado se le añade una capa lineal (768×2) conectada sobre el token $[CLS]$ o $< s >$ (según BERT o RoBERTa)
2. Se entrena todo el modelo con el conjunto de training